

# QTL Mapping - Maintenance Guide

## Operating Environment & Libraries:

- Operating System: Windows 10 or later / macOS / Linux (tested primarily on Windows)
- Python Version: 3.11.9 (recommended)  
Earlier or later Python 3.11 versions are expected to work, but compatibility is not guaranteed below 3.10.
- Hardware:
  - RAM: At least 8GB recommended for running SIM and SMT on medium-large datasets.
  - CPU: Multi-core processor for parallel computations (especially useful for LOD score modelling).
  - Storage: Minimum 500MB free space (to store Results, PDFs, and plots).
- Libraries:
  - numpy
  - pandas
  - scipy
  - matplotlib
  - Pillow
  - Customtkinter

These libraries are declared in the requirements.txt file and can be installed using:

```
pip install -r requirements.txt
```

## Installation Instructions:

Ensure Python 3.11.9 is installed on your system.

You can download it from: <https://www.python.org/downloads/release/python-3119/>

### **Step 1: Clone the Repository:**

Open a terminal and run:

```
git clone https://github.com/Tamert98/QTL-Mapping-Project.git
```

```
cd QTL-Mapping-Project
```

### **Step 2: Install Dependencies:**

Open a terminal and run:

```
pip install -r requirements.txt
```

# **QTL Mapping Project Structure:**

The system is organized into 6 main components:

## **1. Graphical User Interface (GUI) : gui\_frames/**

This folder contains all the CustomTkinter frame modules that build the step-by-step visual workflow:

- file\_format\_window.py: Shows VCF and trait file format instructions.
- file\_selection\_frame.py: Lets the user select input files.
- loading\_data\_frame.py: Loads and parses input data, creates genetic maps.
- sim\_view\_results\_frame.py, smt\_results\_frame.py, smt\_sim\_compare\_frame.py, view\_results\_frame.py:  
Display results of SMT, SIM, and comparison.
- smt\_sim\_runframe.py: Runs both SMT and SIM algorithms after data is prepared.

## **2. Application Entry Point: main\_app.py**

The main **GUI launcher**. It connects all frames and manages transitions.

Key Role:

- Starts at the welcome page.
- Manages file selection, data loading, and flow to SMT/SIM steps.
- Handles callback logic between stages.

## **3. Data Parsing and Preprocessing: vcf\_data\_handler.py**

This is the core data engine that handles:

- VCF parsing (parse\_vcf\_file) — Extracts markers, genotype strings, and quality metrics (GT, PL, GQ, DP).
- Trait file parsing (parse\_trait\_file) — Reads traits per individual.
- Genetic map generation:
  - generate\_combined\_genetic\_maps: Unfiltered distances.
  - generate\_combined\_genetic\_maps\_filtered: Uses genotype quality filters.
- Marker filtering: select\_evenly\_spaced\_markers picks representative SNPs for analysis.

#### 4. Plotting and Visualization: plot\_utils.py

Provides all visualization tools used in the QTL mapping pipeline. It generates plots for genetic maps, recombination heatmaps, SMT significance results, and SIM LOD curves. Also handles exporting visuals as images and PDFs , key functions:

- `generate_genetic_map_images_and_pdf(...)`:
  - Saves each chromosome's genetic map as an image.
- `generate_comparative_genetic_map_images_and_pdf(...)`:
  - Saves distance comparison plots (filtered vs unfiltered).
- `plot_full_recombination_heatmap(...)`:
  - Visualizes recombination rates using dot heatmaps for each chromosome
- `generate_smt_chromosome_images_all_traits(...)`:
  - Generates  $-\log_{10}(\text{p-value})$  significance plots for each trait and chromosome.
- `generate_concatenated_qtl_pdf(...)`:
  - Combines all concatenated SMT trait plots into one multi-page PDF.
- `plot_all_lod_curves(...)`:
  - Plots SIM LOD score curves per chromosome and trait.

#### 5. Single Marker Test: single\_marker\_test.py

Detects significant marker-trait associations using per-marker statistical tests (ANOVA-based F-test or chi-square test for binary traits), key functions:

- `run_smt_for_all_traits(vcf_data, trait_data, send_message)`:
  - Loops through all traits and runs SMT on each.
  - Saves .csv files with columns: chromosome, position, p-value.
- `get_best_marker_info(smt_results, trait)`:
  - Identifies marker with lowest p-value for the given trait.
  - Returns chromosome, position, and p-value.

## 6. Single Interval Mapping: single\_interval\_mapping.py

Performs likelihood-based QTL detection by modelling marker intervals and optimizing the QTL position and effect, key functions:

- `compute_likelihood_params(...)`:
  - Estimates likelihood given a QTL at a fixed position.
  - Returns likelihood scores (L and L0).
- `neg_log_likelihood_sum(...)`:
  - Negative log-likelihood of the full model (QTL affects trait), using P0, P1, and Gaussian modeling of the trait values.
- `neg_log_likelihood_sum0(...)`:
  - Negative log-likelihood of the null model (no QTL effect) assuming a single Gaussian distribution.
- `compute_likelihoods_for_interval_run(...)`:
  - Optimizes both L and L0 using `scipy.optimize.minimize`.
- `run_sim_on_selected_markers(...)`:
  - Iterates over selected marker intervals per chromosome and trait.
  - Computes LOD scores at midpoints between markers.
  - Stores results for visualization.
  - Writes reports to `Results/SIM/Reports/{trait}_report.csv`.